

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 8: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper functions live in `starter_code.py` at the week root: `generate_weighted_intervals`, `generate_knapsack_instance`, `total_weight`, `intervals_compatible`, `selection_is_feasible`.

The 12 problems are grouped into three themes:

- **Weighted Interval Scheduling** (Problems 1–4): computing $p(j)$ by binary search, the bottom-up DP for the optimal value, solution reconstruction, and a top-down memoized version cross-checked against the bottom-up one.
- **Knapsack & Subset Sum** (Problems 5–7): the Boolean subset-sum table, the 2-D 0/1 knapsack value, and item reconstruction.
- **More DP** (Problems 8–12): coin change (minimum coins and number of ways), rod cutting, longest increasing subsequence, and an instrumented comparison of naive recursion vs. memoization vs. bottom-up.

1 Weighted Interval Scheduling

Problem 1 – Compute $p(j)$ via Binary Search

In Weighted Interval Scheduling we sort the n intervals by finish time, so $f_0 \leq f_1 \leq \dots \leq f_{n-1}$. For each interval j ,

$p(j)$ = the index of the latest interval $i < j$ compatible with j , or -1 if none.

Implement `compute_p(intervals)` where each interval is a tuple (`start`, `finish`, `weight`): sort by finish time, then for each j use **binary search** (the `bisect` module) over the finish times to find $p(j)$ in $O(\log n)$. Return the list p indexed by position in the finish-sorted order.

Example. For $[(0,3,1), (1,4,1), (3,5,1), (5,7,1)]$ (already finish-sorted), $p = [-1, -1, 0, 2]$.

Problem 2 – Weighted Interval Scheduling: Optimal Value (Bottom-Up DP)

Implement `weighted_interval_scheduling_value(intervals)` returning the maximum total weight of a compatible subset. Sort by finish time, compute p (Problem 1), and fill the table bottom-up:

```
M[0] = 0
M[j] = max( M[j-1],                # skip interval j
            weight_j + M[p(j) + 1] ) # take interval j
```

(using 1-based M where $M[j]$ is the optimum over the first j intervals). Return $M[n]$. No recursion.

Problem 3 – Reconstruct the Optimal Set of Intervals

Computing the value is not enough – recover the actual intervals. Implement `weighted_interval_scheduling_solution(intervals)` returning (`optimal_value`, `chosen_intervals`), sorted by finish time. After filling M , trace back: interval $j-1$ is in the optimum iff `weight + M[p(j-1)+1] >= M[j-1]`; if so jump to $j = p(j-1)+1$, else $j -= 1$. The chosen set must be feasible and its weights must sum to the value.

Problem 4 – Memoized (Top-Down) Weighted Interval Scheduling

Implement `memoized_wis_value(intervals)` solving the *same* recurrence top-down with memoization (e.g. `functools.lru_cache` or a dict), rather than filling a table. The tests confirm it returns *exactly* the bottom-up value (Problem 2) on many random instances – demonstrating that top-down and bottom-up DP agree.

2 Knapsack & Subset Sum

Problem 5 – Subset Sum: Achievability (Boolean DP)

Given non-negative integers `nums` and a `target`, decide whether some subset sums to exactly `target`. Implement `subset_sum_achievable(nums, target)` with a Boolean DP: `reachable[t]` is True iff some subset sums to t ; start with only $t = 0$ reachable, and for each number x update `reachable` for t from `target` down to x (descending order keeps each item used at most once). Return False for negative targets.

Example. `subset_sum_achievable([3,34,4,12,5,2], 9) == True (=4+5)`; `subset_sum_achievable([3,3,30]) == False`.

Problem 6 – 0/1 Knapsack: Maximum Value (2-D DP)

Each item i has integer weight `weights[i]` and value `values[i]`; the knapsack has integer `capacity`. Each item is taken at most once. Implement `knapsack_value(weights, values, capacity)` via the 2-D DP:

```
dp[0][w] = 0
dp[i][w] = dp[i-1][w]                if weights[i-1] > w
dp[i][w] = max( dp[i-1][w],          # skip item i-1
               values[i-1] + dp[i-1][w-weights[i-1]] ) # take item i-1
```

Return `dp[n][capacity]`. This runs in $O(nW)$ – pseudo-polynomial (see the notes).

Example. `knapsack_value([1,3,4,5], [1,4,5,7], 7) == 9` (items of weight 3+4, value 4+5).

Problem 7 – 0/1 Knapsack: Item Reconstruction

Implement `knapsack_solution(weights, values, capacity)` returning (`best_value`, `chosen_indices`) (ascending indices into the original lists). After building the table, trace back: item $i - 1$ was taken iff `dp[i][w] != dp[i-1][w]`; if so, subtract `weights[i-1]` from w and continue. The chosen items must be distinct, fit within capacity, and have total value equal to the optimum.

3 More DP

Problem 8 – Coin Change: Minimum Number of Coins (Unbounded)

Given coin denominations `coins` (unlimited supply) and a target `amount`, find the minimum number of coins summing to `amount`, or `-1` if impossible. Unlike Week 4's greedy coin change, this DP is correct for *any* coin system. Implement `coin_change_min(coins, amount)`:

```
dp[0] = 0
dp[a] = 1 + min( dp[a-c] for c in coins if c <= a ), else infinity
```

Iterate a from 1 to `amount` *ascending* (coins are reusable, so do NOT reverse the loop, unlike Problem 5). Return `-1` if unreachable.

Example. `coin_change_min([1,2,5], 11) == 3` (5+5+1); `coin_change_min([1,3,4], 6) == 2` (3+3, where greedy would give 3 coins).

Problem 9 – Coin Change: Number of Distinct Ways

Count the distinct *combinations* of coins summing to `amount` (order does not matter: 1+2 and 2+1 are the same). Implement `coin_change_ways(coins, amount)`:

```
ways[0] = 1
for c in coins:          # coin loop OUTSIDE
    for a in c..amount:  # amount loop inside
        ways[a] += ways[a-c]
```

Putting the coin loop outside counts combinations (not permutations). There is exactly one way to make 0 (choose nothing).

Example. `coin_change_ways([1,2,5], 5) == 4`.

Problem 10 – Rod Cutting: Maximum Revenue and the Cut List

Given `prices` where `prices[i]` is the price of a piece of length $i + 1$, cut a rod of length $n = \text{len}(\text{prices})$ to maximize revenue. Implement `rod_cutting(prices)` returning (`best_revenue`, `cuts`):

```
rev[0] = 0
rev[L] = max over k in 1..L of ( prices[k-1] + rev[L-k] )
```

Record, for each L , the first cut length k achieving the maximum, then peel off pieces from $L = n$ downward to build `cuts`. The cut lengths must sum to n and their prices must sum to `best_revenue`.

Example. For `prices = [1,5,8,9,10,17,17,20]` (lengths 1..8), the optimum is 22 (e.g. a length-2 plus a length-6 piece: $5 + 17$).

Problem 11 – Longest Increasing Subsequence ($O(n^2)$ DP)

Implement `longest_increasing_subsequence_length(nums)` returning the length of the longest *strictly* increasing subsequence, via the $O(n^2)$ DP:

$$L[i] = 1 + \max(\{L[j] : j < i, \text{nums}[j] < \text{nums}[i]\} \cup \{0\}), \quad \text{answer} = \max_i L[i].$$

The provided test checks your DP against a brute-force search over all subsequences on small random inputs.

Example. `longest_increasing_subsequence_length([10,9,2,5,3,7,101,18]) == 4`.

Problem 12 – Memoization vs. Bottom-Up: Counting Subproblem Work

This problem makes the central DP idea measurable using the Fibonacci recurrence (the cleanest example of overlapping subproblems). Implement three instrumented functions:

- `naive_fib(n) → (fib(n), call_count)`: plain recursion, counting every call. This grows exponentially – the call count for `fib(n)` is $2 \cdot \text{Fib}(n+1) - 1$.
- `memoized_fib(n) → (fib(n), distinct_subproblems)`: top-down with a cache, counting cache *misses*. This is linear: $n + 1$ for $n \geq 2$.
- `bottom_up_fib(n) → (fib(n), table_writes)`: explicit table, counting writes. Also linear: $n + 1$.

The tests confirm the three values agree, the DP variants do linear work, and naive recursion does $> 100\times$ the work of memoization at $n = 25$.

4 LeetCode Practice

After completing the problems above, practise this week's dynamic-programming techniques (weighted interval scheduling, 0/1 and unbounded knapsack, 1-D DP) on the following [LeetCode](#) problems. Difficulty is LeetCode's own label.

- [Climbing Stairs](#) – *Easy* – 1-D DP.
- [House Robber](#) – *Medium* – 1-D DP.
- [House Robber II](#) – *Medium* – 1-D DP.
- [Coin Change](#) – *Medium* – unbounded knapsack.
- [Coin Change II](#) – *Medium* – counting knapsack.
- [Partition Equal Subset Sum](#) – *Medium* – 0/1 knapsack.
- [Target Sum](#) – *Medium* – 0/1 knapsack.
- [Ones and Zeroes](#) – *Medium* – 2-D knapsack.
- [Maximum Profit in Job Scheduling](#) – *Hard* – weighted interval scheduling.
- [Best Time to Buy and Sell Stock with Cooldown](#) – *Medium* – state-machine DP.